MOHAMED BIN ZAYED UNIVERSITY OF ARTIFICIAL INTELLIGENCE

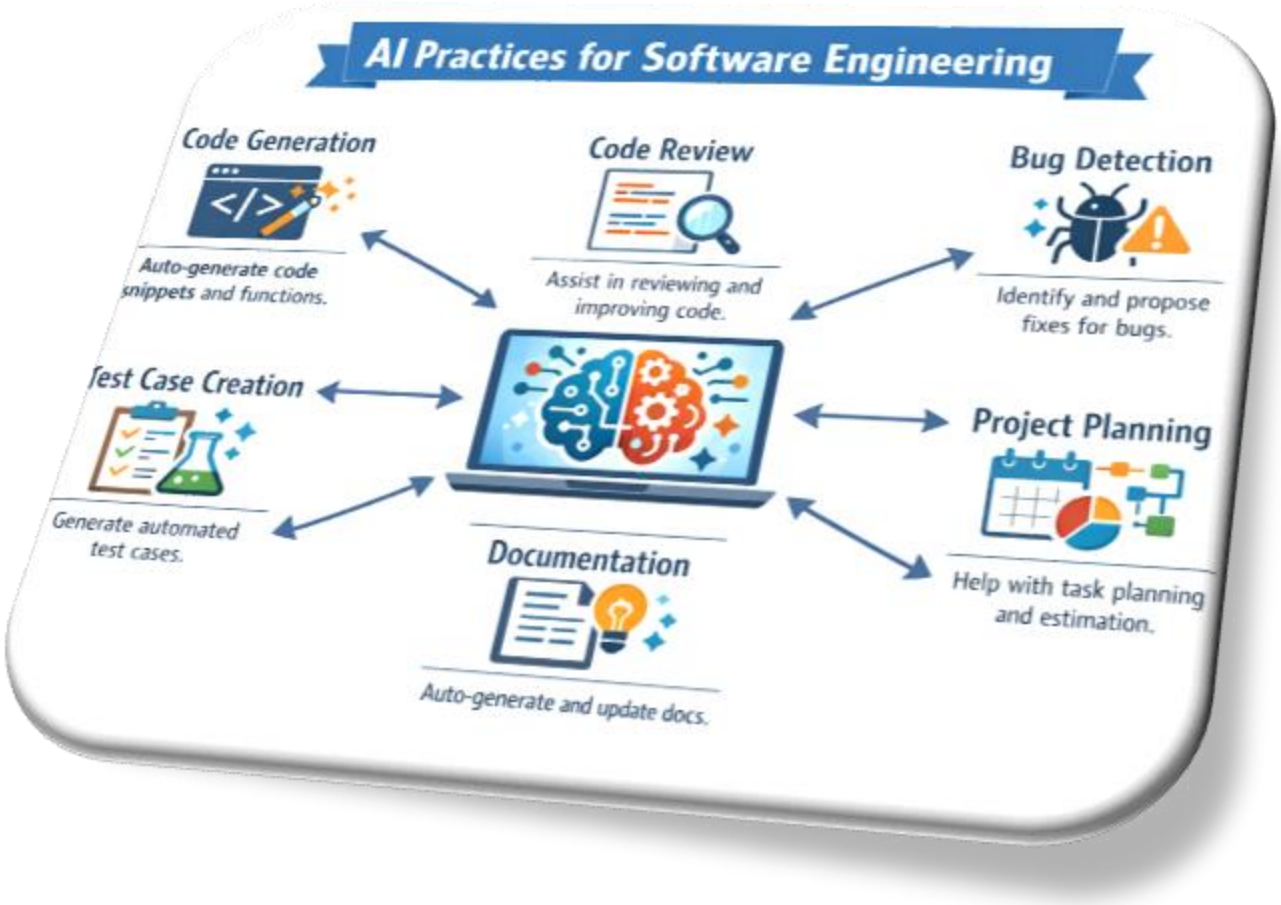
Mohamed bin Zayed University of Artificial Intelligence

SWM Engineering

Date
19/2/2026

6.2

AI Practices for SE



Why AI in Software Engineering?

Context

- Increasing system complexity
- Accelerating release cycles (CI/CD, DevOps)
- Large, distributed codebases
- Rising quality & security expectations

AI's Role

- Augment developer productivity
- Improve software quality
- Reduce operational risk
- Enable intelligent automation

Why AI in Software Engineering?

System	Organization	Approx. Lines of Code	Domain	Notes
Google Monorepo	Google	2+ billion LOC	Web-scale infrastructure	Unified internal repository containing most production systems
Android (AOSP)	Google	~100+ million LOC	Mobile Operating System	Full OS stack including kernel, framework, and system services
Microsoft Windows (modern versions)	Microsoft	~50–70 million LOC (core OS)	Operating System	Core OS components; ecosystem significantly larger
Linux Kernel	Open Source (Linux Foundation)	~30+ million LOC	OS Kernel	Kernel only; excludes full Linux distributions
F-35 Lightning II Software	Lockheed Martin	~8–24+ million LOC	Military Avionics	Mission systems, flight control, sensor fusion, logistics (high safety-critical complexity)

What Do We Mean by “AI Practices”?

AI practices in software engineering include:

- AI-assisted development (code generation, refactoring)
- AI-driven testing and QA
- Intelligent DevOps & AIOps
- ML-enhanced decision-making
- AI-native software architecture
- Governance & responsible AI

AI Across the SDLC

SDLC: Software Development Life Cycle

SDLC Phase	AI Applications
Requirements	NLP-based requirement analysis, ambiguity detection
Design	Architecture recommendation systems
Development	Code completion, automated refactoring
Testing	Test case generation, flaky test detection
Deployment	Predictive deployment risk analysis
Operations	Anomaly detection, incident prediction

What Do We Mean by “AI Practices”?

AI-Assisted Development
Common Capabilities
- Code completion & generation (LLMs)
- Documentation generation
- Static analysis augmentation
- Automated code reviews

Impact
- Reduced development time
- Faster onboarding
- Fewer syntactic defects
- Knowledge amplification

AI in DevOps & AIOps
Intelligent CI/CD
- Predictive build failure detection
- Smart test selection
- Deployment risk scoring

Operations (AIOps)
- Log anomaly detection
- Root cause analysis
- Incident clustering
- Auto-remediation workflows

AI in Testing & Quality Engineering
Common Capabilities
- Automated unit/integration test generation
- Regression test prioritization
- Defect prediction models
- Mutation testing optimization
- Visual regression detection (CV-based)

Impact
Higher coverage
Earlier defect detection
Lower QA cost

MLOps & AI Governance
Operationalizing AI
- Model versioning
- Continuous training (CT)
- Drift detection
- Monitoring & explainability

Governance
- Data privacy compliance
- Bias mitigation
- Secure model deployment
- Responsible AI policies

Risks and Challenges

Risks & Challenges

- Over-reliance on generated code
- Hallucination & correctness issues
- Security vulnerabilities in AI-generated output
- Model drift
- Regulatory constraints
- Intellectual property concerns

Mitigation requires:

- Human-in-the-loop review
- Secure pipelines
- Policy enforcement
- Audit trails

Confirmed Issues from AI in SE

Category of Issue	Confirmed Industry Context	Nature of Error	Observed Impact
Production Bugs from AI Code	Enterprise teams using AI coding assistants	Logic flaws, incorrect implementations	Production defects, rollbacks, increased review effort
Security Vulnerabilities	Security audits of AI-assisted codebases	SQL injection, improper auth handling, unsafe input validation	Exploitable vulnerabilities, increased breach risk
Higher Defect Density	Analyses of real GitHub pull requests	More bugs per LOC compared to human-only code	Increased QA workload, delayed releases
AI-Generated False Bug Reports	cURL open-source project	Fabricated or incorrect vulnerability reports	Bug bounty program discontinued due to overload
Misconfiguration in Deployment Scripts	DevOps environments using AI-generated IaC	Incorrect cloud configs, missing security controls	Service outages, exposed resources
Incomplete Error Handling	Backend services generated via LLMs	Missing edge case handling, silent failures	Runtime instability, customer-facing issues

Let's Focus on Code Generation

Tool / Model	Category	Primary Strength	IDE / Integration	Best For	Key Limitation
OpenAI Codex	Foundational code model	Natural language → code via API	API-based (backend integration)	Custom tooling, embedded AI systems	Not a standalone IDE product
GitHub Copilot	IDE-embedded assistant	Real-time inline completion	VS Code, JetBrains, GitHub	Daily coding productivity	Suggestions require validation
ChatGPT (GPT-4 class models)	Conversational coding assistant	Deep reasoning, debugging, architecture help	Web, API, IDE plugins	Problem-solving & explanation	Less seamless inline editing
Claude (Anthropic)	Long-context code assistant	Repository-level reasoning	CLI, editor integrations	Large codebases, multi-file edits	Smaller IDE footprint vs Copilot
Google Gemini Code Assist	Enterprise coding AI	Large context + cloud integration	VS Code, JetBrains, GCP	Enterprise & cloud-native teams	Strongest inside Google ecosystem
Cursor	AI-native IDE	Codebase chat + multi-file refactor	Standalone AI-enhanced editor	Developers wanting AI-first workflow	Requires switching editors

What is Codex?

Codex is a large language model (LLM) trained to generate and understand code.

- Built on **transformer** architecture (more on this in ML and NLP courses)
- Trained on natural language + publicly available code
- Converts natural language instructions into executable code
- Supports multiple programming languages

Codex Features

Core Capabilities

Codex can:

- Generate functions from English descriptions
- Complete partial code
- Translate between languages
- Refactor and optimize code
- Write unit tests
- Explain existing code

Example prompt:
“Write a Python function that validates an email address.”

How Codex Works

- Input: Natural language or code prompt
- Tokenization into subword units
- Transformer-based attention mechanism
- Next-token probability prediction
- Iterative generation until completion

Key concept: Codex predicts code the same way language models predict text — probabilistically.

Pros and Cons

Strengths

- Accelerates development cycles
- Reduces repetitive coding
- Lowers barrier to entry for new developers
- Multi-language flexibility
- Adaptable via API

Weaknesses

- Can generate incorrect code.
- Struggles with complex logic.
- May introduce security flaws.
- Sensitive to prompt wording.
- Can use outdated practices.
- Requires human review.

Codex augments engineers; it does not replace them.

Codex Best Practices

- Provide precise prompts
- Specify language version and frameworks
- Break complex tasks into smaller steps
- Always validate and test outputs
- Use iterative refinement

Example:

Instead of:
"Build a web app."
Use:
"Create a Flask API in Python 3.11 with JWT authentication and PostgreSQL."

Codex in Action

Installation

Codex CLI can be installed using Homebrew (recommended for macOS) or npm.
Follow the instructions for your operating system.

Installing on macOS with Homebrew

Step 1: Install Homebrew (if not already installed) Homebrew is a package manager for macOS that makes installing software easy.

Open Terminal and run:

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)
```

Follow the prompts. When installation completes, you may need to add Homebrew to your PATH. The installer will show you the exact commands, which typically look like: echo 'eval "$(/opt/homebrew/bin/brew shellenv)"' >> ~/.zprofile eval "$(/opt/homebrew/bin/brew shellenv)"

Verify Homebrew is installed:

```
brew --version
```

Codex in Action

Installation (cont)

```
brew install --cask codex
```

Verify that Codex is installed

```
codex --version
```

You should something like:

```
codex-cli 0.104.0
```

Codex in Action

Running Codex for the first time

```
codex
```

A browser window will open. Sign in with your **MBZUAI account credentials**. After successful authentication, you'll see a confirmation message in your terminal.

Codex in Action

```
>_ OpenAI Codex (v0.104.0)

model:     gpt-5.3-codex   /model to change
directory: ~

Tip: New Build faster with the Codex App. Run 'codex app' or visit https://chatgpt.com/codex?app-
landing-page=true

› Explain this codebase

  ? for shortcuts                                                    100% context left
```

Codex in Action: building a project

AGENTS.md is a configuration file that tells Codex about your project.

It's like a README specifically for your AI assistant.

File Location:
Place AGENTS.md in **your project root directory** **(e.g., myproject)**

Codex in Action: building a project

Computer Architecture Visualizer

Project Overview
A Python tool that detects and visualizes computer hardware specifications including CPU, memory, and storage.

Tech Stack
- Language: Python 3.10+
- psutil: System information gathering
- matplotlib: Graphical visualization
- rich: Terminal formatting

Commands
- Run: `python visualizer.py`
- Test: `pytest tests/`
- Install deps: `pip install -r requirements.txt`

Code Style
- Use type hints for all functions
- Use dataclasses for data structures
- Follow PEP 8 guidelines
- Include docstrings for public functions

Constraints
- Always ask before installing new packages
- Create backups before major file changes

Codex in Action: building a project

We run codex from **myproject** folder and ask "**what is this project about?**"

```
>_ OpenAI Codex (v0.104.0)

model:     gpt-5.2-codex   /model to change
directory: ~/myproject

Tip: New Build faster with the Codex App. Run 'codex app' or visit https://chatgpt.com/codex?app-
landing-page=true

› what is this project about?

• Explored
  └ List ls
    Read AGENTS.md

• It's the Computer Architecture Visualizer: a Python 3.10+ tool that detects and visualizes hardware
  specs (CPU, memory, storage). It uses psutil for system info, matplotlib for graphics, and rich for
  terminal formatting. Run with python visualizer.py, tests via pytest tests/.

› Explain this codebase

  ? for shortcuts                                                    100% context left
```

Codex in Action: prompting

Prompt 1: Outline

"I want to create a Python program that gives info about my computer. First, outline what components we need to detect and how to structure the code. Don't write the code yet."

Codex in Action: prompting

Prompt 2: System Information

"Create a module called **sysinfo.py** that collects:
- CPU: model, cores, frequency, current usage
- Memory: total, available, used, percent
- Disk: available and used space

Use dataclasses for the data structures."

Codex in Action: prompting

Prompt 3: Display

"Create a display module named **display.py** using the rich library that shows the system information in a formatted table with color-coded usage bars. The output should look professional."

Codex in Action: prompting

Prompt 4: Main App

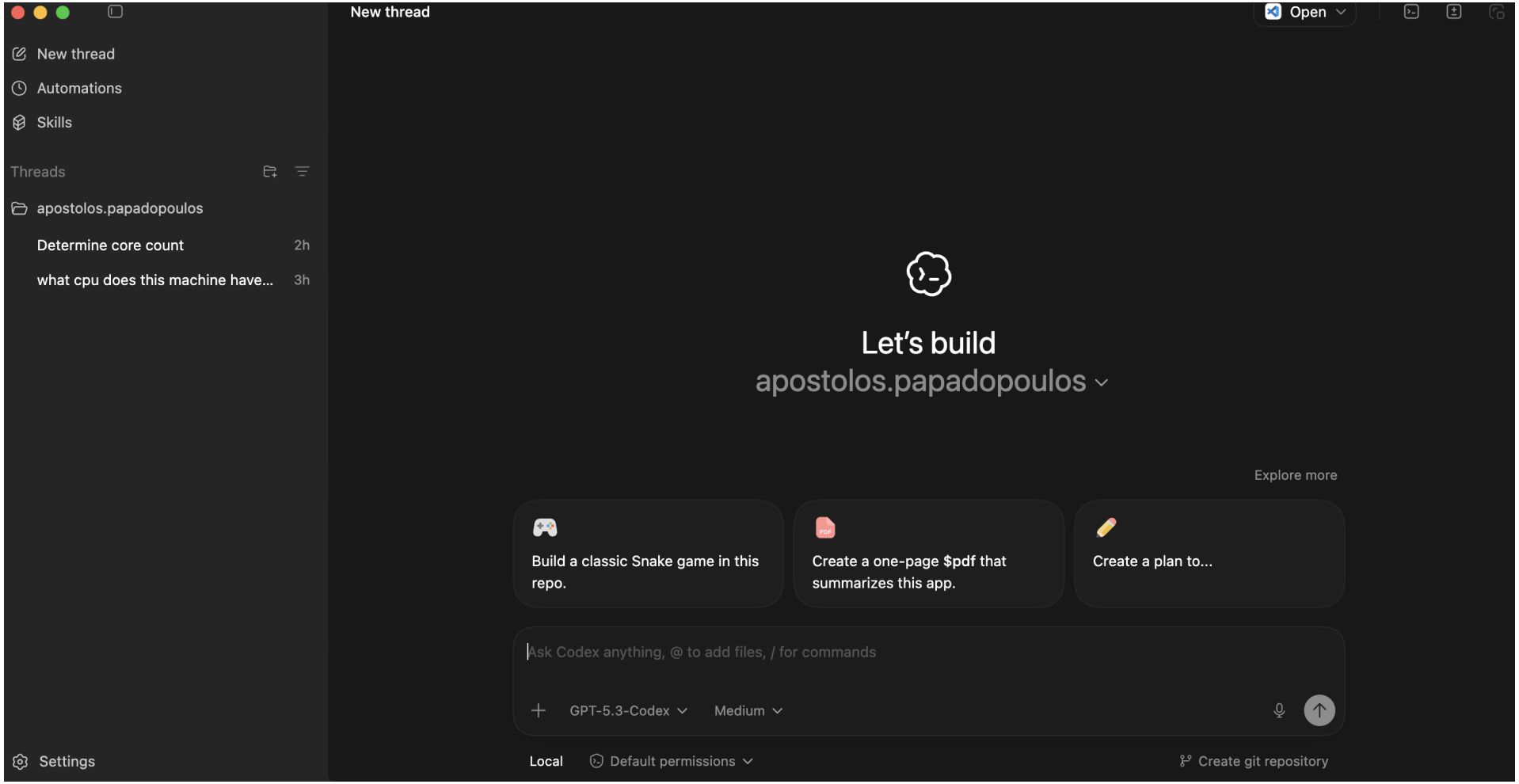
"Create the main **myvisualizer.py** that imports the previous modules and displays a complete system overview. Add a --refresh flag to continuously update the display."

Codex in Action

DEMO TIME !

Codex in Action

Codex App is also available (more user friendly).



Mohamed bin Zayed
University of
Artificial Intelligence

Thank you